CS 327: Compilers

Group-7

Members:

- Chaitanya Sharma 23110072
- Harshil Shah 23110132
- Jeet Joshi 23110148
- Akshat Saurin Shah

Introduction & Compilation Steps

Github repository: https://github.com/HarshilShah1804/FortranPlusPlus
File Structure:

```
FortranPlusPlus/
├── ast/
│   └── (AST construction and related implementation files)
├── lexer/
│   └── lexer.l
├── parser/
│   └── parser.y
    └── grammar_with_conflicts.y
    └── unambiguous_grammar.y
├── references/
│   └── (reference materials, documentation, or grammar notes)
├── tests/
│   └── (test cases for validating the compiler)
├── compile.sh
├── run_parser_tests.sh
├── create_parsing_table.py
└── README.md (optional)
```

The repository contains 3 grammar files:

1. Grammar_with_conflicts.y: The initial design of the grammar without any conflict resolution (Made after Task 1 of assignment)
2. Unambiguous_grammar.y: Conflict-Resolved grammar made after resolving all conflicts while performing the Task 2 of the assignment.
3. Parser.y: Final Grammar with the utilities for printing the reverse derivation tree and error handling (Task 3 and 5 of assignment)

To compile the parser, we have made a script `compile.sh` which would clean any existing parser binary and build it again. If you want to build a parser using any other grammar present, you can replace the file name in the script and remove SET_TOKEN() constructs from the lexer.

To run the set of present tests using the built parser, run the script `run_parser_tests.sh`. It would execute all the tests using the parser binary and show the statistics of the test.

To create the parsing table from the grammar (Task 4), use `yacc` to generate the `y.output` file for the grammar and run the script `create_parsing_table.py`. It would create a parsing table in `.csv` format. Also, a parsing table is created at every run of CI on the repository. So, the latest parsing table can be accessed here.

Part-1: Construction of LALR(1) Automaton

We constructed the LALR(1) parser for our grammar using `yacc`. The parser generates item sets and builds the automaton required for parsing. After generating the automaton with `yacc`, we observed **16 shift-reduce conflicts** and **44 reduce-reduce conflicts**.

The complete automaton contains **318 states**. Since including all states in the report would make it excessively long, the full list can be accessed here.

Conflicts:

Using the `y.output` file generated by `yacc`, we identified the conflicting states, the type of conflict in each state, and the productions responsible for them.

State 0: 2 shift/reduce conflicts:

Productions involved:
declarations → /* empty */
declarations → declaration declarations
declaration → pp_directive
Brief reason: On encountering the lookahead `pp_directive`, which can begin a declaration, the parser has two possible actions: it may either reduce `declarations` to `ε`, or shift in order to parse `pp_directive` as a declaration.

State 27: 2 shift/reduce conflicts:

Productions involved:
declarations → /* empty */
declarations → declaration declarations

Brief reason: On the lookahead `PP_DEFINE` or `PP_UNDEF`, the parser may either reduce `declarations` to ε, thereby ending the declaration list, or shift to continue parsing another declaration.

State 32: 2 shift/reduce conflicts:

Productions involved:
declarations → /* empty */
declarations → declaration declarations
declaration → pp_directive
Brief reason: Inside `program_body`, when the lookahead is `PP_DEFINE` or `PP_UNDEF`, the parser cannot decide whether to reduce `declarations` to ε or shift to parse `pp_directive` as a declaration.

State 33: 2 shift/reduce conflicts:

Productions involved:
declarations → /* empty */
declarations → declaration declarations
declaration → pp_directive
Brief reason: Within `module_body`, on the lookahead `PP_DEFINE` or `PP_UNDEF`, the parser again faces a choice between reducing `declarations` to ε and shifting to parse `pp_directive` as a declaration.

State 63: 1 reduce/reduce conflict:

Productions involved:
declarations → /* empty /
type_def_body → / empty */
Brief reason: On the lookahead `END_TYPE`, both `declarations` and `type_def_body` may reduce to ε at the same point, resulting in two valid reductions.

State 140: 2 shift/reduce conflicts:

Productions involved:
type_def → TYPE DECL_SEP identifier type_def_body END_TYPE
type_def → TYPE DECL_SEP identifier type_def_body END_TYPE identifier
Brief reason: After reading `END_TYPE`, if the next token is `IDENTIFIER`, the parser cannot determine whether it should reduce a completed `type_def` or shift the identifier as an optional trailing name.

State 240: 43 reduce/reduce conflicts:

Productions involved:
func_call → variable_ref LPAREN argument_list RPAREN

array_ref → variable_ref LPAREN argument_list RPAREN
Brief reason: After parsing `variable_ref ( argument_list )`, both productions are complete and have identical right-hand sides. As a result, the parser has two valid reductions and cannot distinguish between them.

State 288: 2 shift/reduce conflicts:

Productions involved:
declarations → /* empty */
declarations → declaration declarations
declaration → pp_directive
Brief reason: Inside a subroutine body, on the lookahead `PP_DEFINE` or `PP_UNDEF`, the parser may either reduce `declarations` to ε or shift to parse another declaration.

State 290: 2 shift/reduce conflicts:

Productions involved:
declarations → /* empty */
declarations → declaration declarations
declaration → pp_directive
Brief reason: Inside a function body, when the lookahead is `PP_DEFINE` or `PP_UNDEF`, the same ambiguity arises between reducing the declaration list to ε and shifting to parse another declaration.

State 314: 2 shift/reduce conflicts:

Productions involved:
declarations → /* empty */
declarations → declaration declarations
declaration → pp_directive
Brief reason: Within a recursive function body, on the lookahead `PP_DEFINE` or `PP_UNDEF`, the parser may either reduce `declarations` to ε or shift to continue parsing declarations.

Part 2: Conflicts Identification and Resolution

The conflicts identified in Part 1 arise due to a small number of underlying ambiguities in the grammar. Instead of treating each conflicting state independently, we classify them into three main categories:

1. Ambiguity between `func_call` and `array_ref`
2. Ambiguity due to empty productions in declaration
3. Ambiguity in type definitions with optional trailing identifiers

1. Function Call v/s Array Reference Conflict:

The grammar contains the following productions:

`func_call → variable_ref LPAREN argument_list RPAREN`
`array_ref` → variable_ref LPAREN argument_list RPAREN

Both productions have identical right-hand sides. Therefore, after parsing x(a, b) the parser cannot determine whether this represents a function call, or an array reference

This results in **reduce/reduce conflicts (State 240)**.

This is an inherent problem in the language itself and cannot be resolved using the syntax alone. So, we club both the `func_call` and `array_ref` in a combine wrapper like `call_or_ref` and we leave for resolution in the semantic analysis stage where we have more information for resolution like symbol table lookup.

New rule in the grammar:

`call_or_ref → variable_ref LPAREN argument_list RPAREN`

After this change, 43 reduce/reduce conflicts are resolved.

2. Ambiguity due to empty declarations:

The grammar allows:

`declarations → /* empty */`
`declarations → declaration declarations`
`declaration` → pp_directive

When the parser encounters tokens like:

PP_DEFINE

it faces two choices:

- Reduce `declarations → ε`
- Shift and parse `pp_directive` as a `declaration`

This causes multiple **shift/reduce conflicts** (States 0, 27, 32, 33, 288, 290, 314).

To resolve this conflict, we restructure the grammar to separate optional and non-empty declarations. Modified Grammar:

declarations_opt → /* empty */

| declarations

declarations → declaration

| declarations declaration

With this structure, the parser no longer confuses an empty declaration list with the beginning of a valid declaration list. For example, in a statement beginning with PP_DEFINE, the parser now directly interprets it as a declaration without ambiguity.

After this modification, 14 shift/reduce conflicts are resolved.

3. Ambiguity in Type Declaration

The grammar allows:

type_def → TYPE ... END_TYPE
type_def → TYPE ... END_TYPE identifier

After encountering END_TYPE the parser cannot decide whether the type definition ends, or an identifier follows (named END TYPE)

This causes **shift/reduce conflicts (State 140, 63)**.

To resolve this conflict, we refactor the grammar to explicitly capture the optional identifier like this:

type_def → TYPE DECL_SEP identifier type_def_body END_TYPE type_name_opt

```
type :: person

  integer :: age

end type

type :: person

  integer :: age

end type person
```

```
type_name_opt → /* empty */
               | identifier
```

After this, the last 3 conflicts are also resolved and now the grammar is completely unambiguous.

Part 3: Reverse Derivation Tree

During the parsing phase, our compiler actively stores the entire derivation tree in memory rather than just validating the syntax. To manage this, we implemented a dedicated modular subsystem (derivation_tree.h and derivation_tree.c) using a DerivationNode structure.

```
typedef struct DerivationNode {
   char *symbol;
   struct DerivationNode **children;
   size_t child_count;
} DerivationNode;
```

Each DerivationNode structure stores the current node symbol and a list of pointers to the children of the Node.

Because our Fortran grammar rules have varying numbers of symbols on the right-hand side, we utilized C's variadic functions (stdarg.h) to build the dt_make_node function. This allows the parser to pass an arbitrary number of child nodes dynamically whenever a reduction occurs.

Example of node creation snippet in parser_with_dt.y:

```
expression: expression ARITH_OP expression
    {
        $$ = NODE("expression", 3, $1, LEAF("ARITH_OP"), $3);
    }
```

In the above example, whenever the parser reduces using the rule `expression: expression ARITH_OP expression`, a derivation-tree node is created for the non-terminal `expression` which is an abstraction for the struct Node defined above, named as `"expression"` and stored with `3` children, the left `expression`, terminal `"ARITH_OP"`, and the right `expression`.

Along with the reverse derivation tree, we also have a flag `--tree` which would print the normal parse tree from top to bottom so that for debugging purposes, it is easier to track any errors since it goes very hand in hand with the original code

Reverse Order Display

To fulfill the requirement of displaying the tree in reverse order, we implemented a custom post-order traversal function (dt_print_postorder). Because LALR(1) is a bottom-up parsing algorithm, performing a strict post-order traversal (visiting children from left to right before printing the parent) exactly mirrors the reverse sequence of the derivations used to parse the input string.

For example, for the below program:

```
program main

end program main
```

We get the following reverse derivation tree:

```
Derivation Tree Output:
        PROGRAM
        IDENTIFIER
          preamble
          declarations_with_modules
          executables
          contains_block
        program_body
        END
        PROGRAM
        IDENTIFIER
      program_unit
    compilation_unit
  compilation_units
start
```

Where each of the `contains_block`, `executables`, `declarations_with_modules` and `preamble` are empty since this is an empty program. So, it represents the sequence of reductions used by the parser for this program.

Part 4: Parsing Table Output

Whenever we create a parser using `yacc`, it generates a file `y.output` file which contains the complete rules for the state transition and actions. So, we created a python script to fetch the important details from the `y.output` file and generate the parsing table from that.

Methodology:

1. The file `y.output` contains the information about terminals, non-terminal and the states.
2. Each state contains the details of actions or goto to be decided upon seeing the next terminal or non-terminal like this:

```
State 4

    1 start: compilation_units •
    4 compilation_units: compilation_units • compilation_unit

    PROGRAM  shift, and go to state 1
    MODULE   shift, and go to state 2

    $default  reduce using rule 1 (start)

    compilation_unit  go to state 13
    program_unit      go to state 7
    module_unit       go to state 8
```

3. We maintain some global trackers like the current state and set of terminals and non terminals.
4. So, the python script reads the `y.output` file line-by-line, and whenever we encounter a line like `State <NUM>`, it marks the current state as `<NUM>` and inside the information of current state, we get all the actions like `reduce`, `goto` etc.
5. So, we use regex to parse the lines of the states to get the current action and store it in a dictionary format. And after completely reading the file, we generate a table, (similar in format to class notes) in `.csv` format which is the complete parsing table of the parser.

Note:

- Since it works as a script, it is very portable and can automatically change the table, whenever we make any changes in the grammar.
- It is also set up in our CI pipeline at the GitHub repository. So, whenever there is a push made to the repository, a fresh parsing table is created and is stored in the run artifacts.

Task 5: Error Handling

As a part of error handling, we display the current line number, and column number of error, the code at that line along with the expected token like this:

```
fprintf(stderr, "\x1b[31mPARSE ERROR\x1b[0m\n");
fprintf(stderr, "Message : %s\n", s);
fprintf(stderr, "Line    : %d\n", yylineno);
fprintf(stderr, "Column  : %d\n", token_column);
fprintf(stderr, "Near    : '%s'\n", near_text);
```

Also, for common fortran errors like missing closing parentheses or missing endif statements after if blocks, we have specific error handling along with special error messages. For implementing it, we use tracker variables which track the number of open parentheses or if blocks. After completing the parse of any block, we have any parenthese or if block open, we throw an error with a specific error message, so that it is easy for the user to identify the issue and fix it.

For example, for the code below:

```
PROGRAM test
 a = (b + 5
END PROGRAM test
```

The parser gives the following error:

```
(base) harshil-shah@harshil-shah-HP-Pavilion-Laptop-15-eg2xxx:~/A2_23110132$ ./parser_out tests/errors/test_error_no_closing_bracker.f
Starting parse
PARSE ERROR
Message : syntax error, unexpected END
Line    : 3
Column  : 1
Near    : 'END'
-> Code: END
         ^
Hint    : Missing closing ')'
Parsing failed.
```

Task 6: Additional Information

We have performed 2 additional tasks other than the mentioned ones, to improve the utility of the compiler and improve debugging:

1. **`--tree`** flag**:** In Task 3, we were required to print the reverse derivation tree generated during parsing. In addition to this, we implemented a `--tree` flag that prints the normal derivation tree from top to bottom using a pre-order traversal of the parse tree. This is especially useful for debugging, because the normal parse tree follows the structure of the original source code more closely. As a result, it becomes easier to trace the parser's behavior step by step and identify incorrectly parsed constructs, which is comparatively more difficult when using only the reverse derivation tree.
2. **Abstract Syntax Tree Creation**: This is an additional feature, different from what was asked in the assignment. We have also created an Abstract Syntax Tree (AST) as an intermediate representation of the parsed program. Unlike the parse tree, which closely follows the grammar and includes all syntactic details, the AST is a simplified and more compact representation that captures only the essential structural and semantic information of the program. This step would further be helpful when we are doing the semantic analysis on the parsed outputs as AST already turns it into a very structured

format. At present, the main branch of the repository contains only the reverse derivation tree implementation. The AST-based parser is available in a separate branch: HarshilShah1804/FortranPlusPlus/ast. To use it, simply run the `compile.sh` script and parse any Fortran program using the generated binary. The execution will also produce a printed version of the AST.